

A Broken Clock Is Right Twice a Day

Gregorian Recurrence, Exact Coincidence, and Business-Day Equivariance

Arian Nazarian*

March 2025



Figure 1: Damaged Omega Speedmaster 3221.30.00 with its civil-time and calendar indications frozen at 10:08:00 on Saturday, 11 March. Its small-seconds counter shares a subdial with a 24-hour hand, so the display distinguishes the two halves of the day [5].

Abstract

A watch frozen at a second-level civil-time and calendar display defines a finite recurrence problem on the proleptic Gregorian calendar. This paper writes the 400-year cycle as an exact translation of the ordinal day index,

$$\tau(\Phi(d)) = \tau(d) + 146\,097,$$

and uses that identity to connect the recurrence of a multifunction watch display, independent calendar implementations, and standard business-day adjustment rules.

For 11 March, the residue

$$h(Y, 3, 11) \equiv Y + C(Y) \pmod{7}$$

gives exactly 56 Saturdays in one 400-year cycle. The pictured Omega Speedmaster 3221.30.00 includes a 24-hour indication, so its frozen state at 10:08:00 on Saturday, 11 March matches exactly

*Ming Hsieh Department of Electrical and Computer Engineering, USC Viterbi School of Engineering, University of Southern California, Los Angeles, CA, USA. Email: ariann@usc.edu.

one second-level state on each qualifying date. Under a uniform random-glance model, the resulting probability is therefore

$$p_{\text{strict}} = \frac{1}{225\,406\,800}.$$

Independent ordinal, Zeller-congruence, and ECMAScript implementations agree over all 146,097 dates in the cycle. The same translation symmetry implies that weekend-only Following, Preceding, and Modified Following adjustments commute with the 400-year shift. For holiday-enriched calendars in which Following is defined throughout, preservation of the business-day indicator is equivalent to Following equivariance and is sufficient for the corresponding Preceding and Modified Following identities. In the weekend-only model, 1,372 of the 4,800 month-ends in one cycle require Modified Following adjustment.

Keywords: Gregorian calendar; day-of-week algorithms; finite-state probability; business-day conventions; calendar equivariance; date algorithms.

MSC 2020: 00A08 (primary); 11A07, 91G80 (secondary).

1 Introduction

The familiar claim that a broken clock is right twice a day is exact only after the display and the sampling rule are specified. A time-only 12-hour dial repeats its reading twice per day, but a multifunction watch can also freeze month, date, weekday, seconds, and a 24-hour state. The problem then becomes one of recurrence in a deterministic finite calendar rather than a product of independent field probabilities.

Behind the calculation is a deliberately fanciful premise. Consider a traveler that is moored in time, bound not by will but by a watch standing still. The hands are stuck, the calendar unchanged, yet this broken mechanism serves as their only guide. They may only journey to the rare intersections where time’s machinery meets its motionless twin. A companion browser visualization¹ uses that premise directly: the frozen display determines the target civil state, and the program reveals the years in which the calendar can reproduce it.

The underlying calendar arithmetic is established. Zeller’s congruence supplies a classical weekday formula [13], and modern treatments such as Dershowitz and Reingold [1] develop Gregorian computation systematically. More recently, Neri and Schneider [4] derive efficient Gregorian conversion algorithms from the same integer structure that makes the 400-year cycle computationally useful. Standard financial date adjustment is likewise existing institutional and software machinery: the 2021 ISDA Definitions specify business-day conventions and an end-of-month convention, while QuantLib implements the corresponding adjustment operators [3, 6, 7].

The present note places these pieces under one translation identity. First, the 400-year Gregorian recurrence is stated as an exact civil-day translation, not only as a repeated weekday table. Second, the Saturday–11 March count is reduced to a direct integer residue and checked exhaustively by independent implementations. Third, the same translation identity is carried through Following, Preceding, and Modified Following, with a precise condition showing when the argument survives the addition of holiday rules and a TARGET-calendar counterexample showing when it does not. The frozen-watch probability is then a finite count inside the same structure rather than a separate calculation.

2 Gregorian Recurrence

The Gregorian calendar was promulgated in 1582 as a reform of the Julian calendar; adoption thereafter varied by jurisdiction.[1, 11] Its leap-year rule is familiar: years divisible by 4 are leap years except century years not divisible by 400. Thus every 400-year block contains 97 leap years and 303 common years.[1, 12]

The corresponding day count is

$$D = 303 \cdot 365 + 97 \cdot 366 = 146\,097 = 20\,871 \cdot 7.$$

The leap-year pattern repeats after 400 years, and the cycle contains an integer number of weeks. Hence every valid Gregorian month–date pair returns to the same weekday after a 400-year shift. This is the recurrence used throughout the paper.

¹Available at arian.ms/work.html#eternal-watch under “The Time Traveller’s Broken Watch.”

An ordinal day index

For a positive civil year Y , define the leap-year indicator

$$\ell(Y) = \begin{cases} 1, & 4 \mid Y \text{ and } (100 \nmid Y \text{ or } 400 \mid Y), \\ 0, & \text{otherwise.} \end{cases}$$

The number of leap years among years $1, \dots, n$ is

$$C(n) = \left\lfloor \frac{n}{4} \right\rfloor - \left\lfloor \frac{n}{100} \right\rfloor + \left\lfloor \frac{n}{400} \right\rfloor.$$

Let

$$(a_1, \dots, a_{12}) = (0, 31, 59, 90, 120, 151, 181, 212, 243, 273, 304, 334),$$

where a_M is the number of days preceding month M in a common year. Then, for a valid date $d = (Y, M, q)$,

$$\tau(d) = 365(Y - 1) + C(Y - 1) + a_M + \ell(Y)\mathbf{1}_{\{M > 2\}} + (q - 1).$$

Thus $\tau(1, 1, 1) = 0$, and $\tau(d)$ is the number of elapsed civil days from 1 January 1 in the proleptic Gregorian calendar.

With Zeller's weekday numbering $0 = \text{Saturday}$, $1 = \text{Sunday}$, $\dots$, $6 = \text{Friday}$, 1 January 1 is Monday in the proleptic Gregorian calendar.[1] Hence

$$h(d) \equiv 2 + \tau(d) \pmod{7}.$$

The 400-year translation symmetry

Let $\mathcal{D}_+$ be the set of valid proleptic Gregorian dates with year $Y \geq 1$, and define

$$\Phi(Y, M, q) = (Y + 400, M, q).$$

Because adding 400 preserves leap-year status, Φ maps valid dates to valid dates. For an integer k with $\tau(d) + k \geq 0$, write $d + k$ for the unique date whose ordinal index is $\tau(d) + k$.

Theorem 2.1 (Gregorian translation symmetry). *For every $d \in \mathcal{D}_+$,*

$$\tau(\Phi(d)) = \tau(d) + 146\,097.$$

Whenever $d + k \in \mathcal{D}_+$,

$$\Phi(d + k) = \Phi(d) + k.$$

Proof. Every block of 400 consecutive Gregorian years contains 97 leap years, so the completed-year contribution to τ increases by

$$400 \cdot 365 + 97 = 146\,097.$$

The within-year month and day contribution is unchanged because Y and $Y + 400$ have the same leap status. This proves the first identity. Applying it to $d + k$ gives

$$\tau(\Phi(d + k)) = \tau(d) + k + 146\,097 = \tau(\Phi(d)) + k.$$

Uniqueness of the ordinal representation gives the second identity. □

Corollary 2.2 (Weekday recurrence). *For every valid date d ,*

$$h(\Phi(d)) = h(d).$$

Proof. Since $146\,097 = 20\,871 \cdot 7$, the theorem changes τ by zero modulo seven. □

This theorem is stronger than a repeated weekday table: the 400-year shift is an exact translation of the ordinal civil-day index. The weekday recurrence used by the watch calculation and the business-calendar identities in Section 6 are consequences of the same symmetry. The direct day-count construction also provides an arithmetic check on the independent implementations below.

3 Zeller's Congruence and Matching Years

Zeller's congruence gives a compact weekday formula for valid Gregorian dates.[13] For a civil date (Y, M, q) , define

$$L = \begin{cases} 1, & M \leq 2, \\ 0, & M \geq 3, \end{cases} \quad Y^* = Y - L, \quad m = M + 12L,$$

so January and February are treated as months 13 and 14 of the previous year. Set

$$K = Y^* \bmod 100, \quad J = \left\lfloor \frac{Y^*}{100} \right\rfloor.$$

Then

$$h \equiv q + \left\lfloor \frac{13(m+1)}{5} \right\rfloor + K + \left\lfloor \frac{K}{4} \right\rfloor + \left\lfloor \frac{J}{4} \right\rfloor - 2J \pmod{7}. \quad (1)$$

The residue uses the same numbering as above: Saturday is 0 and Friday is 6.

An exact count for 11 March

For the date 11 March, the ordinal formula simplifies enough to count the qualifying years directly. Since $a_3 = 59$,

$$\tau(Y, 3, 11) = 365(Y - 1) + C(Y - 1) + 69 + \ell(Y).$$

Using $365 \equiv 1 \pmod{7}$, $69 \equiv 6 \pmod{7}$, and

$$C(Y) = C(Y - 1) + \ell(Y),$$

the weekday residue becomes

$$h(Y, 3, 11) \equiv Y + C(Y) \pmod{7}. \quad (2)$$

Thus 11 March is a Saturday precisely when

$$Y + \left\lfloor \frac{Y}{4} \right\rfloor - \left\lfloor \frac{Y}{100} \right\rfloor + \left\lfloor \frac{Y}{400} \right\rfloor \equiv 0 \pmod{7}.$$

Evaluating this integer residue over the four centuries of one Gregorian cycle gives

Years	1–100	101–200	201–300	301–400	Total
$\#\{\text{Saturday, 11 March}\}$	13	14	14	15	56

with no date-library assumptions. Equation (2) therefore establishes the headline count directly from Gregorian arithmetic; the software audit in Section 4 is an independent verification of the same result.

Civil-date validity is logically separate from the congruence. A query for 29 February must skip non-leap years, and an impossible pair such as 31 April must be rejected before weekday arithmetic is applied. For fixed $(M, q, h_{\text{target}})$, equation (1) is therefore an exact predicate on candidate years. Scanning R years takes $\Theta(R)$ time and constant auxiliary memory apart from the returned matches.

For example, searching 2001–2100 for Saturday, 11 March returns

2006, 2017, 2023, 2028, 2034, 2045, 2051,
2056, 2062, 2073, 2079, 2084, 2090.

The companion visualization uses the same predicate to generate matching years; the 400-year cycle bounds the exact repeating pattern.

4 Independent Implementations and Verification

The recurrence is implemented in two independent ways. The Python version evaluates Zeller's congruence directly; the JavaScript version delegates Gregorian date arithmetic to the standardized ECMAScript `Date` object in UTC.[2] The JavaScript route matches the browser environment of the companion piece, while the Python route provides an arithmetic cross-check.

Feature	Python (Zeller)	JavaScript (UTC Date)
Month convention	March–December as 3–12; January and February as 13 and 14 of the previous year	ECMAScript months 0–11 internally
Weekday convention	0 = Saturday, ..., 6 = Friday	0 = Sunday, ..., 6 = Saturday
Date validity	Explicit Gregorian validity test	Constructed UTC fields checked against the requested date
Century handling	Explicit in K and J	Built into ECMAScript date arithmetic
Role	Independent modular calculation	Browser-oriented implementation

Table 1: Two independent routes to the same Gregorian recurrence.

The UTC construction needs two details that are easy to miss. ECMAScript normalizes out-of-range date components, so successful construction alone does not prove that the requested civil date exists. Also, `Date.UTC` applies the specification’s legacy full-year interpretation to numerical years 0 through 99; `setUTCFullYear` accepts the supplied year directly.[2] The JavaScript routine therefore constructs in UTC with `setUTCFullYear` and verifies the resulting year, month, and day.

The full implementations are supplied as `zellers.py` and `zellers.js`. Both keep civil-date validity separate from weekday matching, so a normalized JavaScript construction or a modular residue cannot turn an impossible date into a valid one.

Complete-cycle audit

The accompanying `verify_calendar.py` and `verify_calendar.js` scripts enumerate every valid date in years 1–400. The audit compares Zeller’s residue with the direct ordinal construction, translates the ECMAScript weekday numbering to the same convention, and computes a common SHA-256 digest of the 146,097-state weekday sequence. It also checks the leap-century boundary at 1900 and 2000 and independently reproduces the 56 occurrences derived in equation (2). All three computational routes agree across the full cycle.

Section 6 uses a separate verifier for the 146,097-day translation and the business-day equivariance identities. Gregorian weekday arithmetic and financial adjustment rules are therefore checked independently.

5 Probability of a Frozen Display

Let $\mathcal{D}_{400}$ be the 146,097 valid dates in one Gregorian cycle and let

$$\mathcal{S} = \{0, 1, \dots, 86\,399\}$$

index second-level states within a modeled civil day. Then

$$\Omega = \mathcal{D}_{400} \times \mathcal{S}, \quad |\Omega| = 146\,097 \times 86\,400 = 12\,622\,780\,800.$$

A “random glance” means a uniform draw from Ω . For a matching event $A \subseteq \Omega$,

$$\mathbb{P}(A) = \frac{|A|}{|\Omega|}.$$

This is a model of displayed civil-time states at second-level resolution, not exact coincidence in continuous physical time. Time zones, daylight-saving transitions, leap seconds, and jurisdiction-specific historical adoption dates are outside this state space; introducing any of them would define a different sampling model without changing the Gregorian recurrence itself.

The familiar time-only case

A frozen 12-hour dial without an AM/PM indicator has two corresponding readings in a 24-hour day. At the three common display resolutions,

Display resolution	Matching states per day	Probability
Hour	2	1/12
Minute	2	1/720
Second	2	1/43 200

A 24-hour indication, or an explicit AM/PM state, removes the factor of two.

The Omega display

The Speedmaster in Fig. 1 has central hour and minute hands, a small-seconds indication, a 24-hour hand, month and weekday windows, and a central date hand.[5] Chronograph elapsed-time counters are excluded from the civil-time match because their state depends on an externally initiated timing operation.

The frozen civil-time-and-calendar state is therefore

$$\boxed{\text{Saturday, 11 March, 10:08:00 with the displayed 24-hour state.}}$$

Within one Gregorian cycle, 11 March is a Saturday 56 times. Because the 24-hour hand distinguishes the two otherwise identical 10:08:00 readings of the central 12-hour indication, exactly one second-level state on each qualifying date matches the frozen display. Hence

$$p_{\text{strict}} = \frac{56}{12\,622\,780\,800} = \boxed{\frac{1}{225\,406\,800}} \approx 4.44 \times 10^{-9}.$$

If time of day is ignored and only the calendar display is checked,

$$p_{\text{calendar}} = \frac{56}{146\,097} \approx 3.83 \times 10^{-4} \approx \frac{1}{2609}.$$

Under independent repeated random glances, the number of glances through the first match is geometric, so its mean is $1/p$. [9] This interpretation applies to independent inspections; it is not a model of continuous observation or deterministic periodic checking.

How non-uniform are Gregorian triples?

The 400-year cycle also gives the complete frequency range for valid (weekday, month, date) triples. Every ordinary month–date occurs 400 times, and its seven weekday counts are always 56, 57, or 58. The 97 occurrences of 29 February are split among weekdays with counts 13, 14, or 15. Consequently the largest and smallest positive triple probabilities are

$$\boxed{p_{\text{max}} = \frac{58}{146\,097}}, \quad \boxed{p_{\text{min}} = \frac{13}{146\,097}}.$$

Impossible dates have probability zero and are not part of the positive minimum. These exact counts are a useful regression target for any implementation that claims full Gregorian recurrence rather than agreement on a handful of examples.

6 Financial Calendar Mechanics

Financial contracts use deterministic date-generation and date-adjustment rules. The 2021 ISDA Interest Rate Derivatives Definitions treat Payment Dates, Period End Dates, Business Day Conventions, and an End-of-month convention; QuantLib implements the corresponding adjustment machinery in its date infrastructure.[3, 6, 7] The analysis here isolates the Gregorian/weekend component first and then identifies the condition that transfers the same symmetry to holiday-enriched calendars.

The translation map Φ and Theorem 1 now supply the structural input for date adjustment. In particular, $\Phi(d+k) = \Phi(d) + k$ preserves every finite civil-day offset, while Corollary 2 preserves weekday.

Weekend-only business-day adjustment

Define

$$B(d) = \begin{cases} 1, & d \text{ is Monday through Friday,} \\ 0, & d \text{ is Saturday or Sunday.} \end{cases}$$

For dates whose adjustment remains in $\mathcal{D}_+$, let

$$F(d) = d + \min\{k \geq 0 : B(d+k) = 1\},$$

$$P(d) = d - \min\{k \geq 0 : B(d-k) = 1\},$$

and

$$MF(d) = \begin{cases} F(d), & \text{month}(F(d)) = \text{month}(d), \\ P(d), & \text{otherwise.} \end{cases}$$

These are the standard Following, Preceding, and Modified Following conventions.[6]

Theorem 6.1 (400-year business-day equivariance). *Under the weekend-only calendar,*

$$F(\Phi(d)) = \Phi(F(d)), \quad P(\Phi(d)) = \Phi(P(d)), \quad MF(\Phi(d)) = \Phi(MF(d))$$

whenever the corresponding adjustments are defined in $\mathcal{D}_+$.

Proof. Theorem 1 gives $\Phi(d+k) = \Phi(d) + k$, and weekday preservation gives

$$B(\Phi(d+k)) = B(d+k).$$

The admissible forward and backward offsets to the first business day are therefore identical before and after the shift. This proves the identities for F and P . The map Φ also preserves calendar month, so the same branch of the Modified Following definition is selected on both sides. $\square$

The same reasoning applies to any nominal date rule that is itself equivariant under Φ . Fixed month-date rules, nth-weekday rules, and calendar end-of-month rules are examples.

Holiday-enriched calendars

Let $H : \mathcal{D}_+ \rightarrow \{0, 1\}$ indicate an additional holiday exclusion and define

$$B_H(d) = B(d)(1 - H(d)).$$

The proof of Theorem 6.1 uses only preservation of the business-day indicator. It therefore carries over whenever

$$B_H(\Phi(d)) = B_H(d) \quad (d \in \mathcal{D}_+).$$

Proposition 6.2 (Holiday-calendar criterion). *Assume the holiday-enriched Following operator F_H is defined for every $d \in \mathcal{D}_+$. Then*

$$F_H(\Phi(d)) = \Phi(F_H(d)) \quad \text{for every } d$$

if and only if

$$B_H(\Phi(d)) = B_H(d) \quad \text{for every } d.$$

Under this condition, the corresponding Preceding and Modified Following identities also hold wherever those operators are defined.

Proof. If B_H is preserved, the admissible forward and backward offsets to the first business day are identical before and after the shift, exactly as in Theorem 6.1. Conversely,

$$B_H(d) = 1 \iff F_H(d) = d.$$

If Following commutes with Φ , then

$$F_H(d) = d \iff F_H(\Phi(d)) = \Phi(d),$$

which is equivalent to $B_H(\Phi(d)) = 1$. Hence the business-day indicator is preserved. $\square$

Because the weekend indicator B is already 400-year equivariant, the stronger and easy-to-check condition

$$H(\Phi(d)) = H(d) \quad (d \in \mathcal{D}_+)$$

is sufficient. It is not necessary: holiday labels on weekends do not change B_H .

This distinction is useful for real calendar libraries. Timeless fixed-date rules and weekday-based observances can satisfy the condition, while rules governed by additional cycles or historical effective dates need not. QuantLib’s TARGET calendar, for example, includes Good Friday, Easter Monday, and historically specific 31 December closures.[8] Gregorian Easter Sunday was 31 March in 2024 but is 21 April in 2424.[10] Both 29 March 2024 and its 400-year shift, 29 March 2424, are Fridays; the former is Good Friday and the latter is not. Thus the TARGET business-day indicator itself fails the 400-year condition. For this date, Following moves 29 March 2024 to 2 April 2024, whereas 29 March 2424 is already a business day, giving a concrete failure of adjustment equivariance.

Exact month-end frequencies

The ISDA End-of-month convention uses the last calendar day of the month when the convention is specified.[3] There are $400 \times 12 = 4,800$ month-ends in one Gregorian cycle. Exhaustive enumeration gives

Weekday	Mon	Tue	Wed	Thu	Fri	Sat	Sun
Month-ends	687	685	685	687	684	688	684

so 1,372 month-ends fall on weekends:

$$\frac{1372}{4800} = \frac{343}{1200} \approx 0.285833.$$

Under Modified Following, a Saturday month-end moves back one day to Friday and a Sunday month-end moves back two days. The complete-cycle shift distribution is therefore

Nominal month-end	Count	Shift
Monday–Friday	3428	0
Saturday	688	−1 day
Sunday	684	−2 days

with no sampling error.

The supplied `financial_calendar.py` reproduces these counts, verifies the 146,097-day translation on all dates in years 1–400, and verifies all three weekend-only business-day identities over one full cycle away from the positive-year boundary.

7 Conclusion

The central object in this problem is the Gregorian translation symmetry

$$\tau(\Phi(d)) = \tau(d) + 146\,097.$$

Because 146,097 is divisible by seven, the shift preserves weekday; because it preserves every finite civil-day offset, it also preserves weekend-only date adjustments. The watch problem and the financial-calendar results are therefore consequences of the same deterministic structure rather than separate applications placed side by side.

For 11 March, the symmetry can be sharpened to the residue

$$h(Y, 3, 11) \equiv Y + C(Y) \pmod{7},$$

which gives exactly 56 Saturdays in one 400-year cycle. Independent direct, Zeller, and ECMAScript computations reproduce that count over all 146,097 dates. For the pictured Omega, the 24-hour indication removes the usual AM/PM duplication, leaving the strict second-level probability

$$p_{\text{strict}} = \frac{1}{225\,406\,800}.$$

The same translation theorem implies equivariance of weekend-only Following, Preceding, and Modified Following. For holiday-enriched calendars, business-day-indicator preservation is equivalent to Following equivariance and is sufficient for the corresponding Preceding and Modified Following identities. The TARGET counterexample shows why Gregorian recurrence alone cannot determine a real institutional calendar. The exact month-end frequencies then provide a complete-cycle benchmark for implementation.

The motivating question therefore remains intact: when can a frozen watch display be right? Following it through the full Gregorian period leads to a finite recurrence, an exact probability, independently auditable software, and a precise criterion for the calendar operators that preserve the same symmetry.

References

- [1] Nachum Dershowitz and Edward M. Reingold. *Calendrical Calculations*. Cambridge University Press, Cambridge, UK, 3 edition, 2008.
- [2] Ecma International. ECMAScript 2026 Language Specification, 17th edition. <https://tc39.es/ecma262/2026/>, 2026. See the Date algorithms in Section 21.4. Retrieved 12 September 2026.
- [3] International Swaps and Derivatives Association. Key changes in the 2021 ISDA interest rate derivatives definitions. <https://www.isda.org/a/BNEgE/Key-Changes-in-the-2021-ISDA-Interest-Rate-Derivatives-Definitions-June-2021.pdf>, June 2021. Published 24 June 2021. Retrieved 12 September 2026.
- [4] Cassio Neri and Lorenz Schneider. Euclidean affine functions and their application to calendar algorithms. *Software: Practice and Experience*, 53(4):937–970, 2023. doi: 10.1002/spe.3172.
- [5] Omega SA. Speedmaster date / day-date 40 mm, reference 3221.30.00. <https://www.omegawatch.es.com/hk/en-hk/watch-omega-speedmaster-date-day-date-chronograph-40-mm-day-date-32213000>, n.d. Product description and source image. Retrieved 12 September 2026. Image modified for illustrative purposes. Omega is a registered trademark of Omega SA.
- [6] QuantLib Project. Business day conventions. <https://github.com/lballabio/QuantLib/blob/master/ql/time/businessdayconvention.hpp>, n.d.. QuantLib source. Retrieved 12 September 2026.
- [7] QuantLib Project. Calendar interface and business-day adjustment. <https://github.com/lballabio/QuantLib/blob/master/ql/time/calendar.cpp>, n.d.. QuantLib source. Retrieved 12 September 2026.
- [8] QuantLib Project. TARGET calendar implementation. <https://github.com/lballabio/QuantLib/blob/master/ql/time/calendars/target.cpp>, n.d.. QuantLib source implementation including Good Friday, Easter Monday, fixed-date holidays, and historically specific 31 December closures. Retrieved 12 September 2026.
- [9] Sheldon M. Ross. *Introduction to Probability Models*. Academic Press, 12 edition, 2019. ISBN 9780128143469.
- [10] Royal Observatory of Belgium. Easter dates from 1583 to 3000. <https://robinfo.oma.be/en/astro-info/calendar/easter-dates-from-1583-to-3000/>, n.d. Gregorian Easter-date table; 31 March 2024 and 21 April 2424. Retrieved 12 September 2026.
- [11] United States Naval Observatory. Introduction to calendars. <https://aa.usno.navy.mil/faq/calendars>, n.d.. Retrieved 12 September 2026.
- [12] United States Naval Observatory. Leap years. https://aa.usno.navy.mil/faq/leap_years, n.d.. Retrieved 12 September 2026.
- [13] Christian Zeller. Kalender-formeln. *Acta Mathematica*, 9:131–136, 1886. doi: 10.1007/BF02406733.